

Name: Monisha Raghini Chelle

RollNo:2520030537

Section-6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

12.To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical  foldername  moni  sample1.c
OSLAB                          Simple-Commands-Week1  hello  monis  sample1.c.save
OSLAB14                        example.txt      hello.c  sample1
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create the Single-Pipe Program

Run:

nano single_pipe.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano single_pipe.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Paste this code:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 single_pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        close(fd[1]);

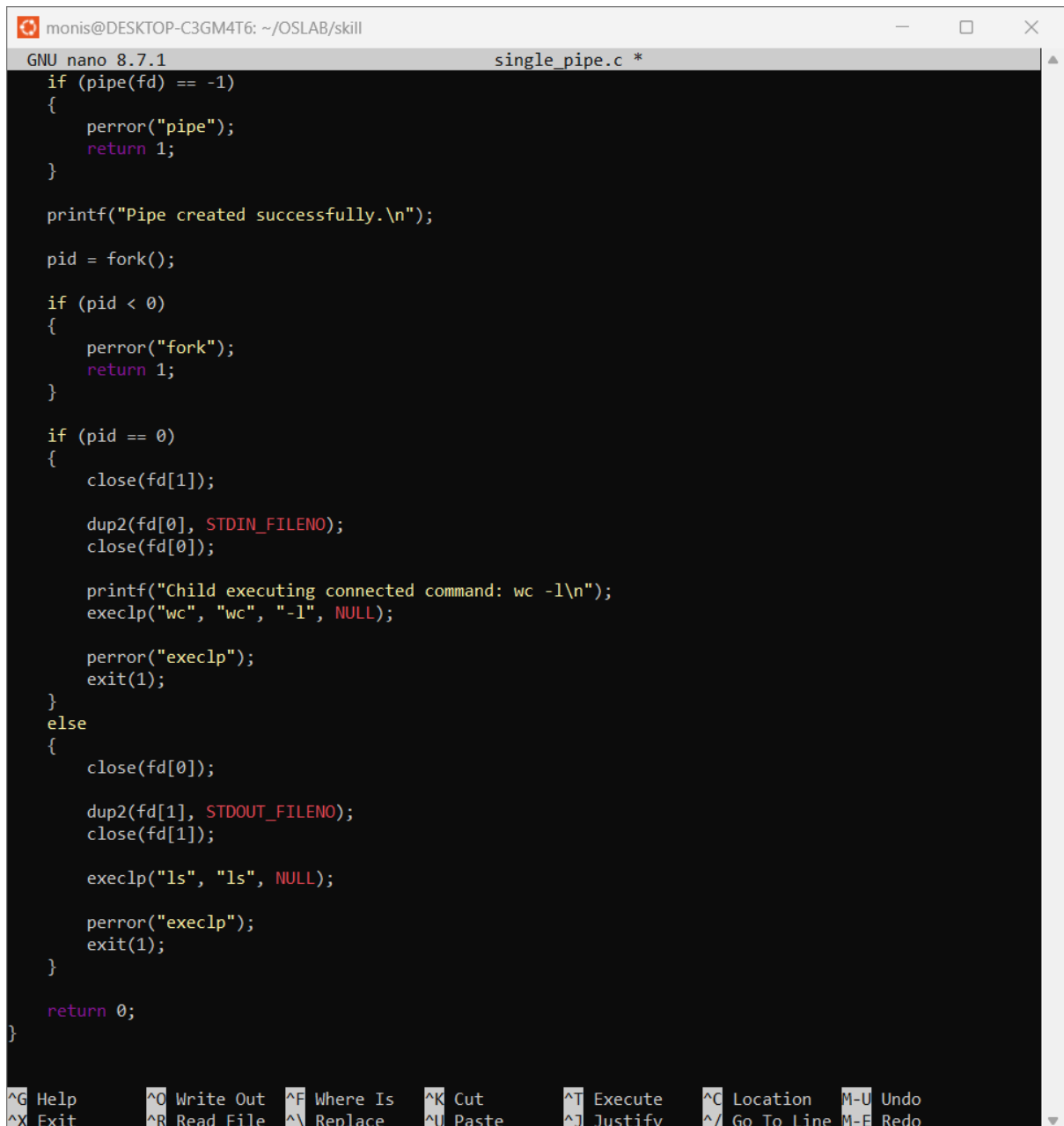
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        printf("Child executing connected command: wc -l\n");
        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        exit(1);
    }
    else
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/ Go To Line	M-E Redo



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 single_pipe.c *
if (pipe(fd) == -1)
{
    perror("pipe");
    return 1;
}

printf("Pipe created successfully.\n");

pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    close(fd[1]);

    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);

    printf("Child executing connected command: wc -l\n");
    execlp("wc", "wc", "-l", NULL);

    perror("execlp");
    exit(1);
}
else
{
    close(fd[0]);

    dup2(fd[1], STDOUT_FILENO);
    close(fd[1]);

    execlp("ls", "ls", NULL);

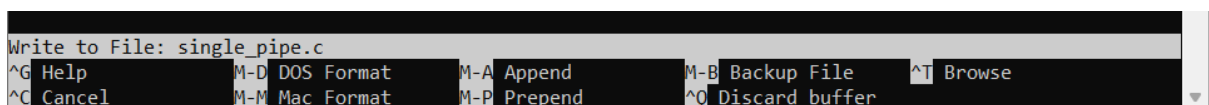
    perror("execlp");
    exit(1);
}

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^N Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save:

Ctrl + O → Enter → Ctrl + X



```
Write to File: single_pipe.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
^C Cancel M-M Mac Format M-P Prepend ^Q Discard buffer
```

Step 3 — Compile the Single-Pipe Program

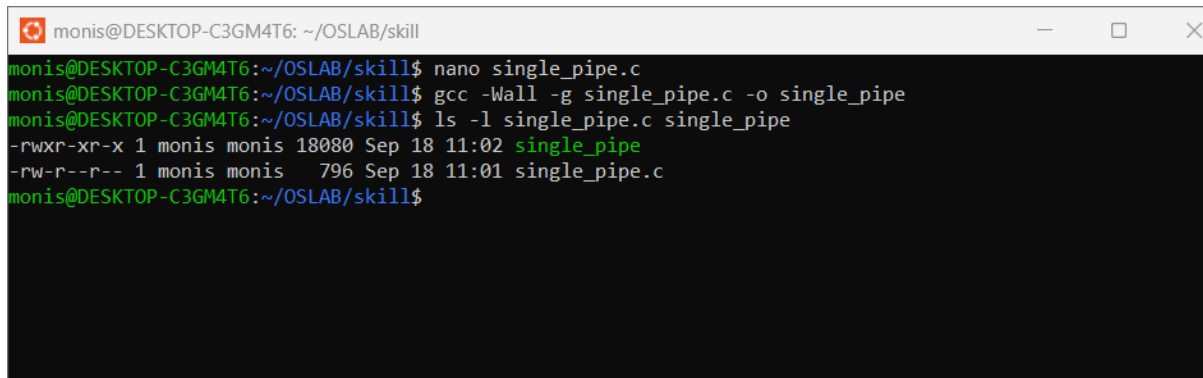
Now compile the program:

```
gcc -Wall -g single_pipe.c -o single_pipe
```

If there are **no error messages**, compilation is successful.

Then verify the files:

```
ls -l single_pipe.c single_pipe
```

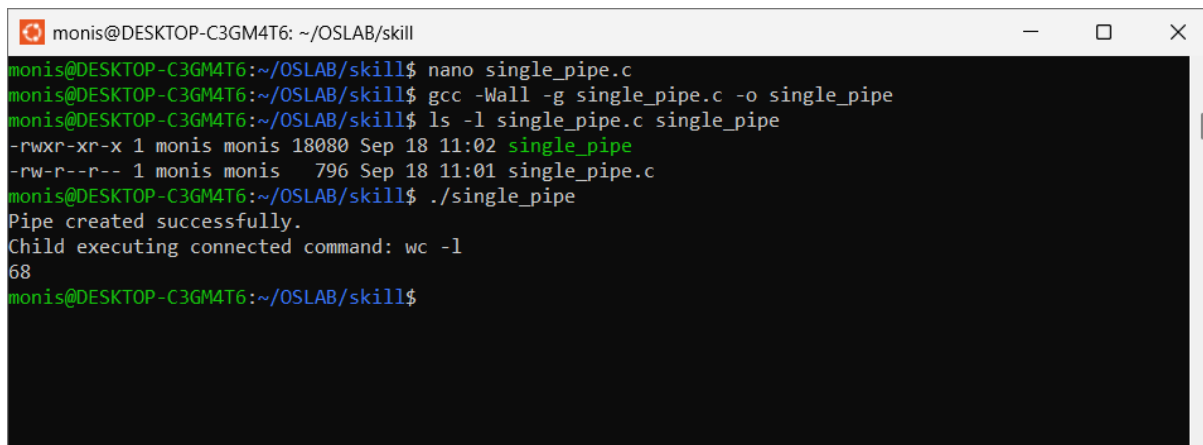
A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill' showing the following commands and output:

```
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano single_pipe.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g single_pipe.c -o single_pipe
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l single_pipe.c single_pipe
-rwxr-xr-x 1 monis monis 18080 Sep 18 11:02 single_pipe
-rw-r--r-- 1 monis monis 796 Sep 18 11:01 single_pipe.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Run the Single-Pipe Program

Now run:

```
./single_pipe
```

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill' showing the following commands and output:

```
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano single_pipe.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g single_pipe.c -o single_pipe
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l single_pipe.c single_pipe
-rwxr-xr-x 1 monis monis 18080 Sep 18 11:02 single_pipe
-rw-r--r-- 1 monis monis 796 Sep 18 11:01 single_pipe.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./single_pipe
Pipe created successfully.
Child executing connected command: wc -l
68
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Create a Synchronization Test

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pipe_sync.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
GNU nano 8.7.1 pipe_sync.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        close(fd[1]);

        printf("Child process waiting for data...\n");

        char buffer[100];

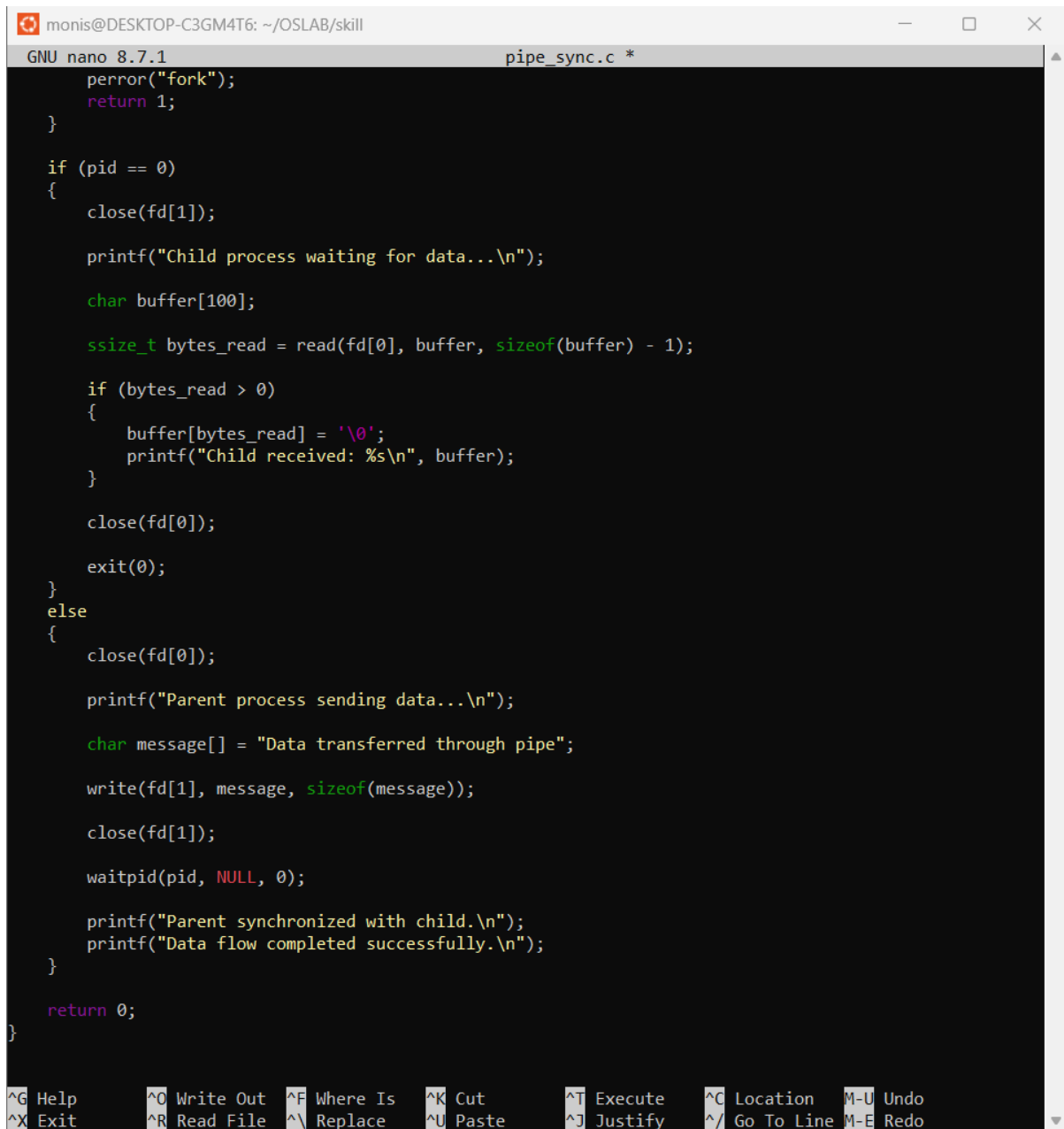
        ssize_t bytes_read = read(fd[0], buffer, sizeof(buffer) - 1);

        if (bytes_read > 0)
        {
            buffer[bytes_read] = '\0';
            printf("Child received: %s\n", buffer);
        }

        close(fd[0]);

        exit(0);
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-F Redo



```
monis@DESKTOP-C3GM4T6: ~/.OSLAB/skill
GNU nano 8.7.1 pipe_sync.c *
    perror("fork");
    return 1;
}

if (pid == 0)
{
    close(fd[1]);

    printf("Child process waiting for data...\n");

    char buffer[100];

    ssize_t bytes_read = read(fd[0], buffer, sizeof(buffer) - 1);

    if (bytes_read > 0)
    {
        buffer[bytes_read] = '\0';
        printf("Child received: %s\n", buffer);
    }

    close(fd[0]);

    exit(0);
}
else
{
    close(fd[0]);

    printf("Parent process sending data...\n");

    char message[] = "Data transferred through pipe";

    write(fd[1], message, sizeof(message));

    close(fd[1]);

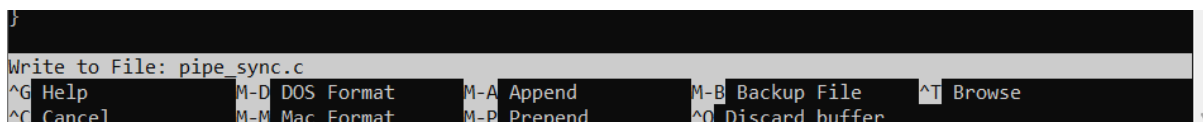
    waitpid(pid, NULL, 0);

    printf("Parent synchronized with child.\n");
    printf("Data flow completed successfully.\n");
}

return 0;
}
```

Save:

Ctrl + O → Enter → Ctrl + X



```
Write to File: pipe_sync.c
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  ^M-U Undo
^X Exit      ^R Read File ^F Replace   ^U Paste     ^J Justify   ^/ Go To Line ^M-E Redo
```

Step 6 — Compile the Synchronization Program

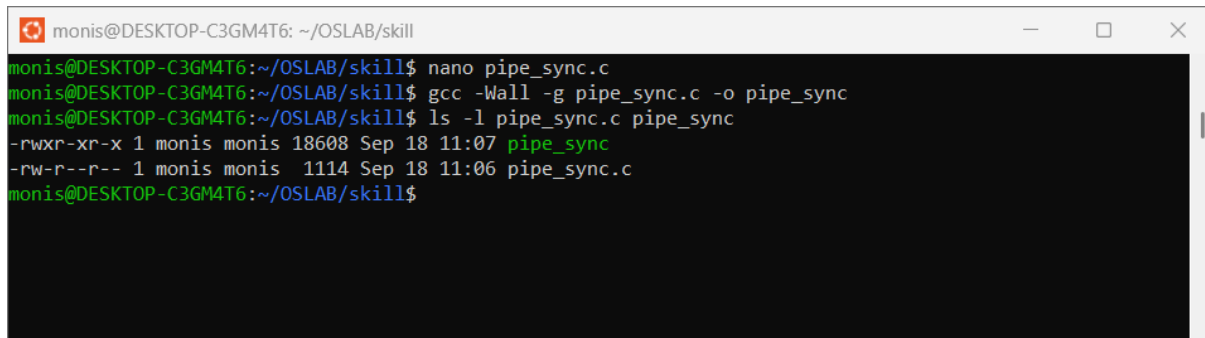
Run:

```
gcc -Wall -g pipe_sync.c -o pipe_sync
```

If there are no error messages, compilation is successful.

Then check:

```
ls -l pipe_sync.c pipe_sync
```

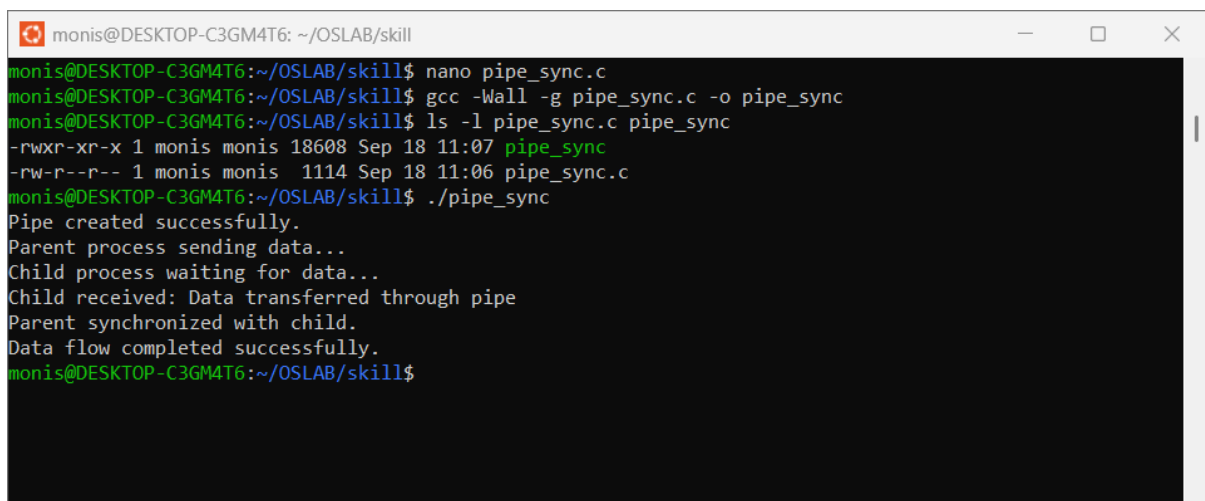


```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pipe_sync.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g pipe_sync.c -o pipe_sync
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l pipe_sync.c pipe_sync
-rwxr-xr-x 1 monis monis 18608 Sep 18 11:07 pipe_sync
-rw-r--r-- 1 monis monis 1114 Sep 18 11:06 pipe_sync.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 7 — Run the Synchronization Test

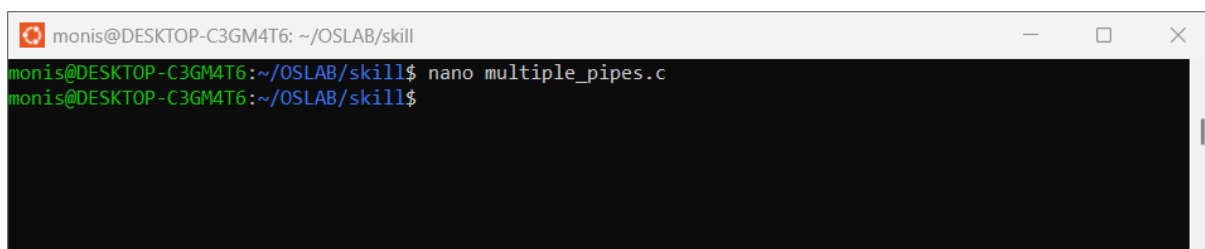
Now run:

```
./pipe_sync
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano pipe_sync.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g pipe_sync.c -o pipe_sync
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l pipe_sync.c pipe_sync
-rwxr-xr-x 1 monis monis 18608 Sep 18 11:07 pipe_sync
-rw-r--r-- 1 monis monis 1114 Sep 18 11:06 pipe_sync.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./pipe_sync
Pipe created successfully.
Parent process sending data...
Child process waiting for data...
Child received: Data transferred through pipe
Parent synchronized with child.
Data flow completed successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 8 — Create the Multiple-Pipe Program



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano multiple_pipes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 multiple_pipes.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipe1[2];
    int pipe2[2];

    pid_t pid1;
    pid_t pid2;
    pid_t pid3;

    if (pipe(pipe1) == -1)
    {
        perror("pipe1");
        return 1;
    }

    if (pipe(pipe2) == -1)
    {
        perror("pipe2");
        return 1;
    }

    printf("Two pipes created successfully.\n");

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(pipe1[0]);
        close(pipe2[0]);
        close(pipe2[1]);

        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[1]);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 multiple_pipes.c *
    close(pipe1[1]);

    execlp("ls", "ls", NULL);

    perror("execlp ls");
    exit(1);
}

pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    close(pipe1[1]);
    close(pipe2[0]);

    dup2(pipe1[0], STDIN_FILENO);
    dup2(pipe2[1], STDOUT_FILENO);

    close(pipe1[0]);
    close(pipe2[1]);

    execlp("sort", "sort", NULL);

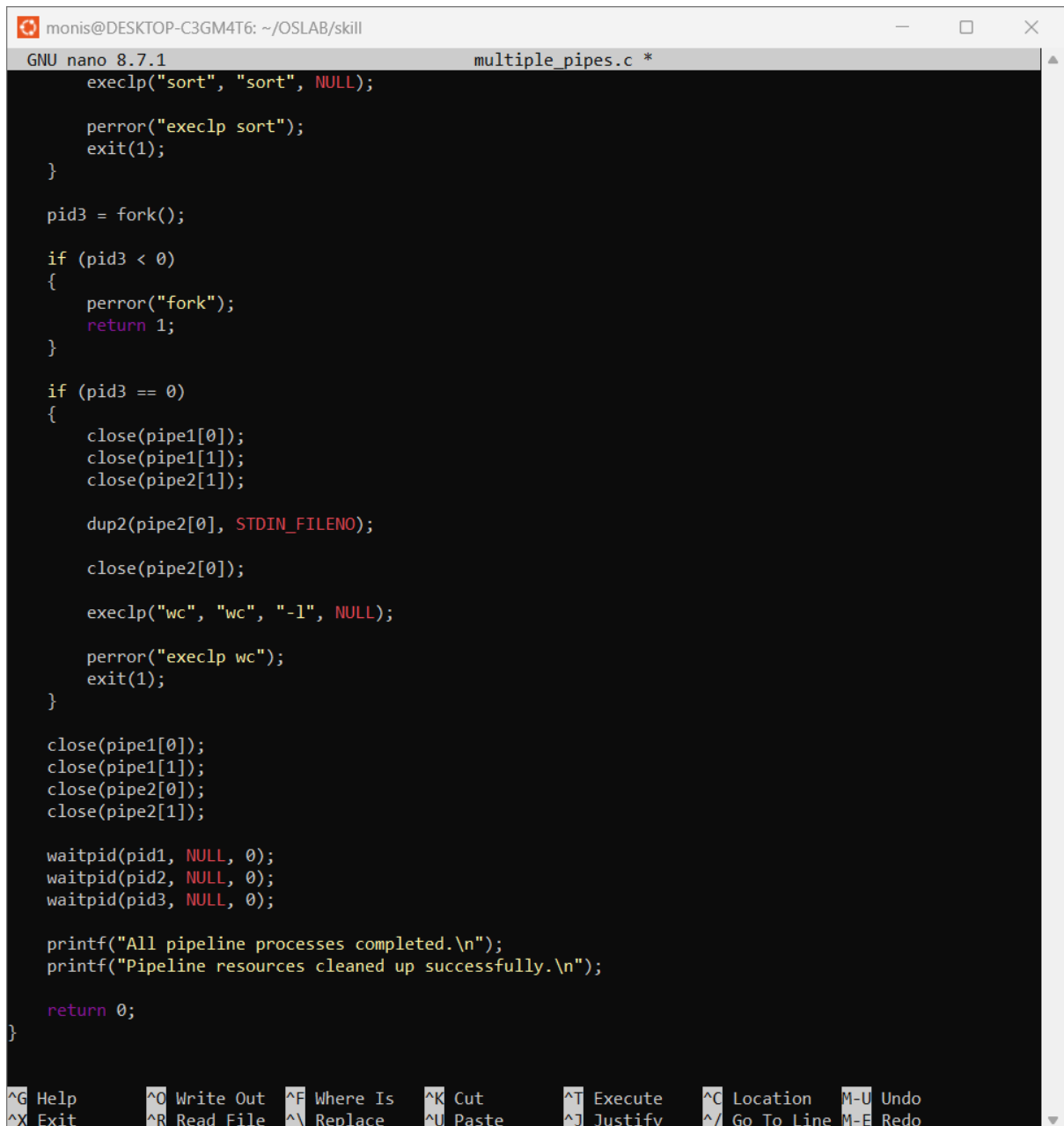
    perror("execlp sort");
    exit(1);
}

pid3 = fork();

if (pid3 < 0)
{
    perror("fork");
    return 1;
}

if (pid3 == 0)
{
    close(pipe1[0]);
    close(pipe1[1]);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 multiple_pipes.c *
    execlp("sort", "sort", NULL);

    perror("execlp sort");
    exit(1);
}

pid3 = fork();

if (pid3 < 0)
{
    perror("fork");
    return 1;
}

if (pid3 == 0)
{
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);

    dup2(pipe2[0], STDIN_FILENO);

    close(pipe2[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("execlp wc");
    exit(1);
}

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
waitpid(pid3, NULL, 0);

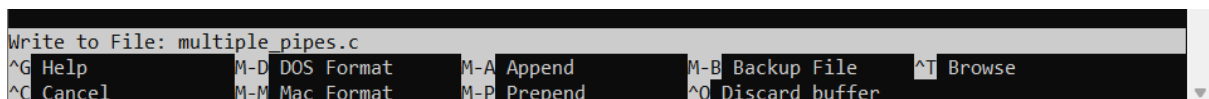
printf("All pipeline processes completed.\n");
printf("Pipeline resources cleaned up successfully.\n");

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save:

Ctrl + O → Enter → Ctrl + X



```
Write to File: multiple_pipes.c
^G Help   M-D DOS Format   M-A Append   M-B Backup File   ^T Browse
^C Cancel   M-M Mac Format   M-P Prepend   ^O Discard buffer
```

Step 9 — Compile the Multiple-Pipe Program

Run:

```
gcc -Wall -g multiple_pipes.c -o multiple_pipes
```

If there are no error messages, compilation is successful.

Then verify:

ls -l multiple_pipes.c multiple_pipes

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano multiple_pipes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g multiple_pipes.c -o multiple_pipes
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l multiple_pipes.c multiple_pipes
-rwxr-xr-x 1 monis monis 18352 Sep 18 11:16 multiple_pipes
-rw-r--r-- 1 monis monis 1789 Sep 18 11:14 multiple_pipes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 10 — Run the Multiple-Pipe Program

Now run:

./multiple_pipes

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano multiple_pipes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g multiple_pipes.c -o multiple_pipes
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l multiple_pipes.c multiple_pipes
-rwxr-xr-x 1 monis monis 18352 Sep 18 11:16 multiple_pipes
-rw-r--r-- 1 monis monis 1789 Sep 18 11:14 multiple_pipes.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./multiple_pipes
Two pipes created successfully.
72
All pipeline processes completed.
Pipeline resources cleaned up successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 11 — Test a Long Pipeline

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano long_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
GNU nano 8.7.1 long_pipeline.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipe1[2];
    int pipe2[2];
    int pipe3[2];

    pid_t pid1, pid2, pid3, pid4;

    if (pipe(pipe1) == -1 ||
        pipe(pipe2) == -1 ||
        pipe(pipe3) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Three pipes created successfully.\n");

    pid1 = fork();

    if (pid1 == 0)
    {
        close(pipe1[0]);
        close(pipe2[0]);
        close(pipe2[1]);
        close(pipe3[0]);
        close(pipe3[1]);

        dup2(pipe1[1], STDOUT_FILENO);
        close(pipe1[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 == 0)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
if (pid2 == 0)
{
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe3[0]);
    close(pipe3[1]);

    dup2(pipe1[0], STDIN_FILENO);
    dup2(pipe2[1], STDOUT_FILENO);

    close(pipe1[0]);
    close(pipe2[1]);

    execlp("sort", "sort", NULL);

    perror("sort");
    exit(1);
}

pid3 = fork();

if (pid3 == 0)
{
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);
    close(pipe3[0]);

    dup2(pipe2[0], STDIN_FILENO);
    dup2(pipe3[1], STDOUT_FILENO);

    close(pipe2[0]);
    close(pipe3[1]);

    execlp("grep", "grep", ".c", NULL);

    perror("grep");
    exit(1);
}

pid4 = fork();

if (pid4 == 0)
{
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 long_pipeline.c *
    close(pipe3[1]);

    execlp("grep", "grep", ".c", NULL);

    perror("grep");
    exit(1);
}

pid4 = fork();

if (pid4 == 0)
{
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
    close(pipe3[1]);

    dup2(pipe3[0], STDIN_FILENO);
    close(pipe3[0]);

    execlp("wc", "wc", "-l", NULL);

    perror("wc");
    exit(1);
}

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);
close(pipe3[0]);
close(pipe3[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
waitpid(pid3, NULL, 0);
waitpid(pid4, NULL, 0);

printf("Long pipeline completed successfully.\n");
printf("All descriptors closed and resources cleaned up.\n");

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save:

Ctrl + O → Enter → Ctrl + X

```
Write to File: long_pipeline.c
^G Help   M-D DOS Format   M-A Append   M-B Backup File   ^T Browse
^C Cancel   M-M Mac Format   M-P Prepend   ^O Discard buffer
```

Step 12 — Compile the Long Pipeline Program

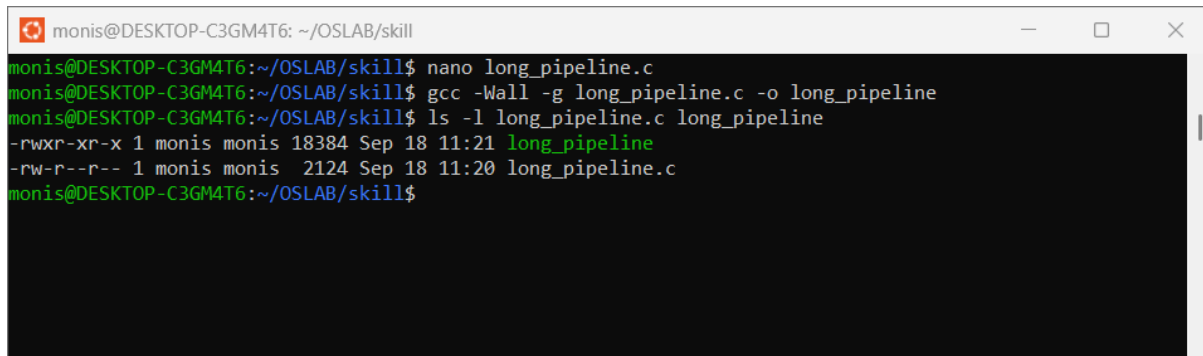
Run:

```
gcc -Wall -g long_pipeline.c -o long_pipeline
```

If there are no error messages, compilation is successful.

Then verify:

```
ls -l long_pipeline.c long_pipeline
```

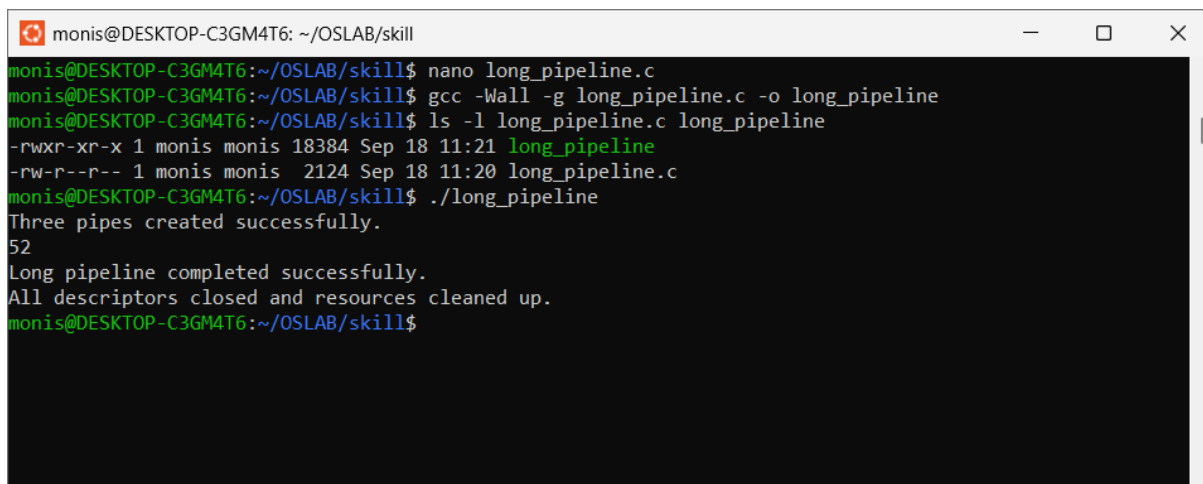
A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The user has executed the following commands: 'nano long_pipeline.c', 'gcc -Wall -g long_pipeline.c -o long_pipeline', and 'ls -l long_pipeline.c long_pipeline'. The output of the last command shows two files: 'long_pipeline' with permissions '-rwxr-xr-x', size 18384, and 'long_pipeline.c' with permissions '-rw-r--r--', size 2124. Both files were created on Sep 18 at 11:21 and 11:20 respectively.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano long_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g long_pipeline.c -o long_pipeline
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l long_pipeline.c long_pipeline
-rwxr-xr-x 1 monis monis 18384 Sep 18 11:21 long_pipeline
-rw-r--r-- 1 monis monis  2124 Sep 18 11:20 long_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 13 — Run the Long Pipeline

Now run:

```
./long_pipeline
```

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSLAB/skill'. The user has executed the same commands as in the previous screenshot, followed by './long_pipeline'. The output shows 'Three pipes created successfully.', '52', 'Long pipeline completed successfully.', and 'All descriptors closed and resources cleaned up.'.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano long_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g long_pipeline.c -o long_pipeline
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l long_pipeline.c long_pipeline
-rwxr-xr-x 1 monis monis 18384 Sep 18 11:21 long_pipeline
-rw-r--r-- 1 monis monis  2124 Sep 18 11:20 long_pipeline.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./long_pipeline
Three pipes created successfully.
52
Long pipeline completed successfully.
All descriptors closed and resources cleaned up.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```